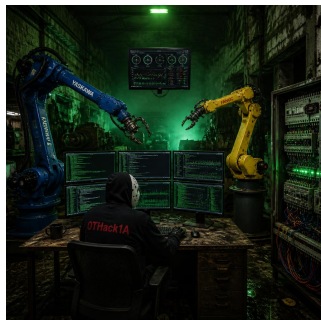




Claude Code CLI

Guida di Riferimento Completa

Comandi, CLI, Hooks, Memoria, Skill, MCP

Edizione Giugno 2026

Fonti: documentazione ufficiale Anthropic (code.claude.com/docs)

OTHack1A

Documento personale ad uso educativo – nessun contenuto generato da IA dichiarato nel testo

Contents

1	Introduzione e metodo di verifica	3
2	Installazione e autenticazione	3
2.1	Metodi di installazione	3
2.2	Comandi di autenticazione	3
3	Avvio di una sessione e flag principali	3
3.1	Modalità di avvio	4
3.2	Flag CLI più rilevanti per uso quotidiano	4
4	La cartella .claude: dove vive la configurazione	4
5	Memoria: CLAUDE.md e Auto Memory	5
5.1	CLAUDE.md	5
5.2	Auto Memory	5
6	Comandi: gestione di sessione e contesto	6
7	Modello, effort e modalità di esecuzione	6
8	Permessi e modalità operative	7
9	Skill e comandi personalizzati	7
9.1	Skill vs comandi legacy	7
9.2	Frontmatter rilevante	7
9.3	Comandi di gestione	7
9.4	Skill incluse (bundled)	8
10	Hooks: automazione sugli eventi del ciclo di vita	8
10.1	Eventi principali	8
10.2	Esempio: formattazione automatica dopo una modifica	8
11	MCP – Model Context Protocol	9
11.1	Aggiungere un server	9
11.2	Comandi di sessione	9
11.3	Risorse e prompt MCP	9
11.4	Tool search	9
12	Git, Pull Request e revisione del codice	9
13	Subagent, background e lavoro parallelo	11
13.1	Gestione da shell	11
14	Sessioni remote, web e dispositivi	11
15	Diagnostica, plugin e personalizzazione interfaccia	11
15.1	Diagnostica	12
15.2	Plugin	12
15.3	Interfaccia	12
16	Claude Code in VS Code	12
16.1	Differenze pratiche rispetto alla CLI pura	12
16.2	Comandi utili specifici per l'integrazione IDE	13

17	Glossario	13
18	Fonti consultate	13

1 Introduzione e metodo di verifica

Questa guida raccoglie in italiano i comandi, le opzioni CLI e i meccanismi di configurazione di Claude Code attivi a giugno 2026. È pensata come riferimento di consultazione, non come tutorial passo-passo.

[i] Metodo

Ogni comando elencato in questo documento è stato verificato sulla documentazione ufficiale Anthropic (code.claude.com/docs) consultata a giugno 2026. Dove un comando citato in guide di terze parti (blog, articoli, repository GitHub non ufficiali) non è risultato presente nella documentazione ufficiale, è stato escluso da questa guida e riportato invece nel documento separato “*Comandi non verificati*”, con la fonte originale, perché l’utente possa testarlo direttamente.

[!] Disponibilità variabile

Non tutti i comandi sono visibili a tutti gli utenti: la disponibilità dipende dal piano (Pro/Max/Team/Enterprise/API), dal sistema operativo e dalla versione installata di Claude Code. Alcuni comandi richiedono una versione minima della CLI, indicata dove nota.

2 Installazione e autenticazione

2.1 Metodi di installazione

```
# Binario nativo (consigliato)
curl -fsSL https://claude.ai/install.sh | bash

# Tramite npm (qualsiasi OS)
npm install -g @anthropic-ai/claude-code

# Verifica versione
claude --version
```

2.2 Comandi di autenticazione

Comando	Funzione
<code>claude auth login</code>	Accesso con account Anthropic. Opzioni: <code>-email</code> , <code>-sso</code> , <code>-console</code> (per fatturazione API invece di abbonamento)
<code>claude auth logout</code>	Disconnessione dall’account
<code>claude auth status</code>	Stato di autenticazione in JSON (<code>-text</code> per output leggibile)
<code>claude update</code>	Aggiorna alla versione più recente
<code>claude install [versione]</code>	Installa o reinstalla il binario nativo; accetta una versione specifica, <code>stable</code> o <code>latest</code>

Fonte: code.claude.com/docs/en/cli-reference

3 Avvio di una sessione e flag principali

3.1 Modalità di avvio

Comando	Funzione
<code>claude</code>	Avvia una sessione interattiva
<code>claude "query"</code>	Avvia con un prompt iniziale
<code>claude -p "query"</code>	Esegue una query non interattiva (modalità SDK/script) e termina
<code>cat file claude -p "query"</code>	Elabora contenuto passato in pipe
<code>claude -c</code>	Continua la conversazione più recente nella directory corrente
<code>claude -r "<sessione>" "query"</code>	Riprende una sessione per ID o nome

3.2 Flag CLI più rilevanti per uso quotidiano

Flag	Descrizione
<code>-model</code>	Imposta il modello per la sessione (alias <code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code> , oppure nome completo)
<code>-effort</code>	Livello di ragionamento per la sessione: <code>low</code> , <code>medium</code> , <code>high</code> , <code>xhigh</code> , <code>max</code>
<code>-permission-mode</code>	Modalità di permesso iniziale: <code>default</code> , <code>acceptEdits</code> , <code>plan</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code>
<code>-add-dir</code>	Aggiunge directory di lavoro accessibili a Claude (non importa automaticamente la configurazione <code>.claude/</code> di quella directory)
<code>-mcp-config</code>	Carica server MCP da file o stringa JSON
<code>-worktree, -w</code>	Avvia in un git worktree isolato, utile per lavorare su un branch senza toccare la directory principale
<code>-bg</code>	Avvia la sessione come agente in background e ritorna immediatamente il controllo del terminale
<code>-dangerously-skip-permissions</code>	Salta tutte le richieste di permesso (equivalente a <code>-permission-mode bypassPermissions</code>); da usare con consapevolezza
<code>-max-turns</code>	Limita il numero di turni agentici (solo modalità <code>-p</code>)
<code>-max-budget-usd</code>	Limite di spesa in dollari prima di interrompere (solo modalità <code>-p</code>)
<code>-append-system-prompt</code>	Aggiunge testo personalizzato al system prompt di default, mantenendo le istruzioni base
<code>-system-prompt</code>	Sostituisce interamente il system prompt di default (per agenti non di coding)

[i] Nota

`claude -help` non elenca tutti i flag disponibili: la loro assenza dall'help non implica che non esistano. La lista completa è in [cli-reference](#).

Fonte: code.claude.com/docs/en/cli-reference

4 La cartella .claude: dove vive la configurazione

Claude Code legge configurazione, memoria, skill, hook e comandi da due posizioni: la cartella di progetto `.claude/` (nella root del repository) e la cartella personale `~/.claude/` (valida per tutti i progetti).

Percorso	Contenuto
CLAUDE.md	Istruzioni persistenti di progetto, caricate a ogni sessione
CLAUDE.local.md	Istruzioni persistenti locali, non condivise via git
.claude/settings.json	Impostazioni di progetto (permessi, hook, variabili d'ambiente)
.claude/settings.local.json	Impostazioni locali, tipicamente escluse da git (es. <code>claudeMdExcludes</code>)
.claude/skills/<nome>/SKILL.md	Skill personalizzate (formato consigliato, sostituisce i comandi legacy)
.claude/commands/*.md	Comandi personalizzati legacy (formato precedente alle skill)
.claude/agents/*.md	Configurazioni di subagent personalizzati
.claude/rules/*.md	Regole modulari, alternativa a un CLAUDE.md monolitico
~/.claude/CLAUDE.md	Memoria personale valida su tutti i progetti

Fonte: code.claude.com/docs/en/claude-directory, [/en/memory](https://code.claude.com/docs/en/memory), [/en/skills](https://code.claude.com/docs/en/skills)

5 Memoria: CLAUDE.md e Auto Memory

5.1 CLAUDE.md

È un file markdown con istruzioni persistenti che Claude legge per intero a ogni avvio di sessione. Non è una configurazione vincolante: Claude la tratta come contesto e tenta di seguirla, senza garanzia assoluta di rispetto rigoroso, specialmente per istruzioni vaghe o in conflitto tra loro.

[i] Best practice

Mantenere il CLAUDE.md di progetto breve e stabile (indicativamente 20–120 righe): comandi di build, convenzioni, struttura del progetto, regole sempre valide. Spostare procedure articolate o regole specifiche di un sottosistema in skill dedicate o in file sotto `.claude/rules/`.

I commenti HTML a blocco (`<!-- nota -->`) dentro un CLAUDE.md vengono rimossi prima di entrare nel contesto del modello: utili per lasciare annotazioni per i collaboratori umani senza consumare token.

5.2 Auto Memory

Meccanismo che permette a Claude di accumulare conoscenza tra sessioni senza scrittura manuale: annota comandi di build, intuizioni di debug, preferenze di stile, abitudini di workflow, basandosi su correzioni e osservazioni fatte durante il lavoro. È attiva di default da Claude Code v2.1.59. Si gestisce con `/memory`, oppure tramite l'impostazione `autoMemoryEnabled` nei file di progetto.

I file di auto memory risiedono in `~/.claude/projects/<progetto>/memory/` e sono markdown modificabili o eliminabili liberamente.

Fonte: code.claude.com/docs/en/memory

6 Comandi: gestione di sessione e contesto

Comando	Funzione
<code>/clear [nome]</code>	Nuova conversazione a contesto vuoto; la precedente resta disponibile in <code>/resume</code> . Alias: <code>/reset</code> , <code>/new</code>
<code>/compact [istruzioni]</code>	Riassume la conversazione corrente per liberare contesto, mantenendo la stessa sessione
<code>/context [all]</code>	Visualizza l'uso del contesto come griglia colorata, con suggerimenti di ottimizzazione
<code>/resume [sessione]</code>	Riprende una conversazione per ID o nome, oppure apre un selettore. Alias: <code>/continue</code>
<code>/branch [nome]</code>	Crea un ramo della conversazione corrente per esplorare una direzione diversa senza perdere l'originale
<code>/fork <direttiva></code>	Avvia un subagent in background che eredita la conversazione e lavora in parallelo su una direttiva (richiede v2.1.161+)
<code>/rewind</code>	Riporta indietro conversazione e/o codice a un checkpoint precedente. Alias: <code>/checkpoint</code> , <code>/undo</code>
<code>/btw <domanda></code>	Domanda rapida laterale che non si aggiunge alla cronologia principale
<code>/diff</code>	Visualizzatore interattivo delle modifiche non confermate e dei diff per ogni turno
<code>/export [file]</code>	Esporta la conversazione corrente come testo semplice
<code>/copy [N]</code>	Copia l'ultima risposta (o l'N-esima più recente) negli appunti
<code>/rename [nome]</code>	Rinomina la sessione corrente
<code>/recap</code>	Genera un riassunto della sessione corrente su richiesta
<code>/cd <percorso></code>	Sposta la sessione in una nuova directory di lavoro mantenendo la cache di prompt (richiede v2.1.169+)
<code>/add-dir <percorso></code>	Aggiunge una directory di lavoro per l'accesso ai file nella sessione corrente

Fonte: code.claude.com/docs/en/commands

7 Modello, effort e modalità di esecuzione

Comando	Funzione
<code>/model [modello]</code>	Cambia modello e lo salva come default per le nuove sessioni. Senza argomento apre un selettore; tasto s per cambiare solo per la sessione corrente
<code>/effort [livello auto]</code>	Imposta il livello di ragionamento: low , medium , high , xhigh , max , ultracode (combina xhigh con orchestrazione automatica di workflow); auto ripristina il default
<code>/fast [on off]</code>	Attiva/disattiva la modalità rapida
<code>/advisor [modello off]</code>	Attiva un secondo modello di supporto consultato nei momenti chiave di un task (richiede v2.1.98+)

[i] Nota

Il cambio di modello o effort tramite `/model` e `/effort` ha effetto immediato, senza attendere il completamento della risposta in corso.

Fonte: code.claude.com/docs/en/commands, [/en/model-config](https://code.claude.com/docs/en/model-config)

8 Permessi e modalità operative

Comando	Funzione
<code>/permissions</code>	Dialogo interattivo per regole di tipo allow/ask/deny sui tool. Alias: <code>/allowed-tools</code>
<code>/plan [descrizione]</code>	Entra in modalità piano: Claude descrive l'approccio prima di eseguire qualsiasi modifica
<code>/sandbox</code>	Attiva/disattiva la modalità sandbox (piattaforme supportate)
<code>/fewer-permission-prompts</code>	Skill che analizza le trascrizioni per individuare chiamate Bash/MCP di sola lettura ricorrenti e propone una allowlist in <code>settings.json</code>

Le modalità di permesso disponibili via `-permission-mode` sono: **default** (richiede conferma a ogni azione), **acceptEdits** (accetta automaticamente le modifiche ai file), **plan** (pianifica prima di agire), **auto** (classificatore in background, research preview su piani Team/Enterprise/API), **dontAsk**, **bypassPermissions** (nessuna richiesta).

Fonte: code.claude.com/docs/en/commands, [/en/permission-modes](https://code.claude.com/docs/en/permission-modes)

9 Skill e comandi personalizzati

9.1 Skill vs comandi legacy

Le **skill** (`.claude/skills/<nome>/SKILL.md`) sono il formato consigliato e sostituiscono i vecchi comandi in `.claude/commands/*.md`, che restano supportati per compatibilità. Una skill è un prompt strutturato che Claude può invocare sia manualmente con `/nome-skill`, sia autonomamente quando il contesto della conversazione corrisponde alla sua descrizione.

9.2 Frontmatter rilevante

Campo	Effetto
<code>name</code>	Diventa il nome dello <code>/slash-command</code>
<code>description</code>	Usata da Claude per decidere quando invocare la skill autonomamente
<code>disable-model-invocation: true</code>	Solo l'utente può invocarla (utile per azioni con effetti collaterali: deploy, invio messaggi)
<code>user-invocable: false</code>	Solo Claude può invocarla (conoscenza di sfondo non azionabile come comando)
<code>allowed-tools</code>	Concede accesso a tool specifici senza richiesta di permesso quando la skill è attiva
<code>argument-hint</code>	Suggerimento di argomenti mostrato nell'autocompletamento
<code>model</code>	Forza un modello specifico per quella skill (es. haiku per task meccanici e a basso costo)
<code>context: fork / agent</code>	Esegue la skill in un subagent dedicato invece che nella conversazione principale

9.3 Comandi di gestione

<code>/skills</code>	Elenca le skill disponibili; tasto t per ordinare per numero di token, Spazio per nascondere una dal menu
<code>/reload-skills</code>	Ri-scansiona le directory di skill e comandi senza riavviare la sessione (v2.1.152+)

9.4 Skill incluse (bundled)

Alcune skill sono incluse di default in ogni sessione (disattivabili con `disableBundledSkills`): `/code-review`, `/batch`, `/debug`, `/loop`, `/claude-api`, `/run`, `/verify`, `/run-skill-generator`, `/simplify`, `/deep-research` (quest'ultima è un *workflow*, non una skill in senso stretto: distribuisce ricerche web in parallelo e produce un report con fonti).

Fonte: code.claude.com/docs/en/skills, [/en/commands](https://code.claude.com/docs/en/commands)

10 Hooks: automazione sugli eventi del ciclo di vita

Gli hook sono comandi shell, endpoint HTTP o prompt LLM definiti dall'utente, eseguiti automaticamente in punti precisi del ciclo di vita di una sessione. Si configurano in `.claude/settings.json`.

10.1 Eventi principali

Evento	Quando si attiva
PreToolUse	Prima che un tool venga eseguito; può bloccare la chiamata
PostToolUse	Dopo l'esecuzione di un tool (es. formattazione automatica dopo una modifica)
UserPromptExpansion	Quando un comando slash digitato dall'utente si espande in un prompt, prima che arrivi a Claude; utile per bloccare comandi specifici o iniettare contesto
SessionStart	All'avvio della sessione
Stop	Quando la sessione (o un task in background) termina

10.2 Esempio: formattazione automatica dopo una modifica

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write $CLAUDE_FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

Le variabili d'ambiente disponibili agli hook includono `CLAUDE_FILE_PATH`, `CLAUDE_TOOL_NAME`, `CLAUDE_TOOL_INPUT`, `CLAUDE_SESSION_ID`, `CLAUDE_PROJECT_DIR`.

[i] Nota

Un hook `PreToolUse` sul tool Skill si attiva solo quando Claude invoca la skill internamente: se l'utente digita direttamente `/nome-skill`, quel percorso viene intercettato invece dall'evento `UserPromptExpansion`.

Comando di gestione: `/hooks` mostra la configurazione hook attiva per la sessione corrente.

Fonte: code.claude.com/docs/en/hooks

11 MCP – Model Context Protocol

MCP è lo standard apertosi per collegare Claude Code a strumenti esterni: database, tracker di issue, API, browser. Un server MCP espone un insieme di tool che Claude può richiamare durante la sessione.

11.1 Aggiungere un server

```
# Da CLI
claude mcp add <nome> <comando> [argomenti...]

# Da configurazione JSON
claude mcp add-json <nome> '{"command": "npx", "args": ["-y", "@server/pkg"]}'

# Elenco server installati
claude mcp list

# Rimozione
claude mcp remove <nome>
```

11.2 Comandi di sessione

<code>/mcp</code>	Senza argomenti apre l'elenco interattivo dei server; reconnect <server> per riconnettere, enable/disable per cambiare stato
-------------------	--

11.3 Risorse e prompt MCP

I server MCP possono esporre **risorse**, richiamabili con @ come se fossero file, e **prompt**, che diventano comandi nel formato `/mcp__<server>__<prompt>`, scoperti dinamicamente alla connessione.

11.4 Tool search

Per non saturare il contesto con le definizioni di centinaia di tool, Claude Code usa di default un meccanismo di *tool search*: all'avvio della sessione carica solo nomi dei tool e istruzioni del server, recuperando le definizioni complete solo quando un task le richiede effettivamente.

[i] Nota

Quando l'output di un tool MCP supera 10.000 token, Claude Code mostra un avviso. Il limite massimo predefinito è 25.000 token, regolabile con la variabile d'ambiente `MAX_MCP_OUTPUT_TOKENS`.

Fonte: code.claude.com/docs/en/mcp

12 Git, Pull Request e revisione del codice

[!] Correzione rispetto a guide precedenti

Non esistono comandi builtin `/commit` o `/pr` nella documentazione ufficiale attuale. La creazione di commit e pull request avviene tramite richiesta in linguaggio naturale (“commit le modifiche con un messaggio descrittivo”, “crea una PR per questa feature”): Claude usa direttamente i tool Bash/git sottostanti. Un comando `/commit` personalizzato può comunque essere creato come skill propria.

Comando	Funzione
<code>/diff</code>	Mostra le modifiche non confermate (git diff interattivo)
<code>/code-review [livello] [-fix] [-comment] [target]</code>	Verifica il diff corrente per errori di correttezza, riuso, semplificazione, efficienza. <code>-fix</code> applica le correzioni, <code>-comment</code> le posta come commenti inline su una PR GitHub, <code>ultra</code> esegue una revisione multi-agente in cloud
<code>/simplify [target]</code>	Revisione di sola pulizia (riuso, semplificazione, efficienza, livello di astrazione) senza ricerca di bug, applicata direttamente. Su versioni precedenti a v2.1.154 equivale a <code>/code-review -fix</code>
<code>/review [PR]</code>	Revisione locale di una pull request
<code>/security-review</code>	Analizza le modifiche pendenti sul branch corrente per vulnerabilità di sicurezza (injection, problemi di autenticazione, esposizione dati)
<code>/install-github-app</code>	Configura l'app Claude per GitHub Actions su un repository
<code>/autofix-pr [prompt]</code>	Avvia una sessione cloud che monitora la PR del branch corrente e applica correzioni quando la CI fallisce o arrivano commenti dei revisori (richiede CLI gh)
<code>/ultrareview [PR]</code>	Revisione approfondita multi-agente in sandbox cloud; alias preferito ora <code>/code-review ultra</code>

Fonte: code.claude.com/docs/en/commands, [/en/code-review](https://code.claude.com/docs/en/code-review)

13 Subagent, background e lavoro parallelo

Comando	Funzione
<code>/agents</code>	Gestione delle configurazioni di subagent
<code>/tasks</code> (alias <code>/bashes</code>)	Visualizza e gestisce tutto ciò che è in esecuzione in background
<code>/background [prompt]</code>	Stacca la sessione corrente per farla proseguire come agente in background, liberando il terminale. Alias: <code>/bg</code>
<code>/batch <istruzione></code>	Skill che scompone un cambiamento su larga scala in 5–30 unità indipendenti, ciascuna eseguita da un subagent in un git worktree isolato, con apertura automatica di una pull request per unità (richiede repository git)
<code>/fork <direttiva></code>	Vedi sezione contesto: subagent in background che eredita la conversazione
<code>/goal [condizione clear]</code>	Imposta un obiettivo: Claude continua a lavorare nei turni successivi finché la condizione non è soddisfatta
<code>/loop [intervallo] [prompt]</code>	Esegue un prompt ripetutamente mentre la sessione resta aperta
<code>/schedule [descrizione]</code>	Crea, aggiorna o elenca routine pianificate eseguite su infrastruttura cloud gestita da Anthropic. Alias: <code>/routines</code>

13.1 Gestione da shell

<code>claude agents background</code>	# apre la vista agenti (monitoraggio sessioni)
<code>claude agents --json script</code>	# elenco sessioni attive in JSON, utile per script
<code>claude attach <id></code>	# si collega a una sessione in background
<code>claude stop <id></code>	# ferma una sessione in background
<code>claude respawn <id></code>	# riavvia una sessione mantenendo la conversazione
<code>claude logs <id></code>	# stampa l'output recente di una sessione

Fonte: code.claude.com/docs/en/commands, [/en/cli-reference](https://code.claude.com/docs/en/cli-reference), [/en/agent-view](https://code.claude.com/docs/en/agent-view)

14 Sessioni remote, web e dispositivi

Comando	Funzione
<code>/teleport</code>	Importa nel terminale locale una sessione avviata su Claude Code on the web. Alias: <code>/tp</code> . Richiede abbonamento claude.ai
<code>/remote-control</code>	Rende la sessione corrente controllabile da claude.ai o dall'app Claude. Alias: <code>/rc</code>
<code>/desktop</code>	Continua la sessione corrente nell'app Claude Code Desktop (macOS/Windows). Alias: <code>/app</code>
<code>/mobile</code>	Mostra un QR code per scaricare l'app mobile. Alias: <code>/ios</code> , <code>/android</code>
<code>/web-setup</code>	Collega l'account GitHub a Claude Code on the web tramite le credenziali locali gh

Fonte: code.claude.com/docs/en/commands

15 Diagnostica, plugin e personalizzazione interfaccia

15.1 Diagnostica

<code>/doctor</code>	Diagnostica installazione e impostazioni; tasto f per far correggere a Claude i problemi rilevati
<code>/debug [descrizione]</code>	Attiva il log di debug per la sessione corrente e ne analizza il contenuto
<code>/feedback [report]</code>	Invia un feedback o segnala un bug con il contesto della sessione allegato. Alias: <code>/bug</code> , <code>/share</code>
<code>/heapdump</code>	Scriva uno snapshot dell'heap JavaScript per diagnosticare un uso anomalo di memoria
<code>/release-notes</code>	Changelog interattivo con selettore di versione
<code>/insights</code>	Report sulle proprie sessioni: aree di progetto, pattern di interazione, punti di frizione

15.2 Plugin

<code>/plugin [sottocomando]</code>	Gestisce i plugin Claude Code (<code>list</code> , <code>install</code> , <code>enable</code> , <code>disable</code>)
<code>/reload-plugins [-force]</code>	Ricarica i plugin attivi senza riavviare la sessione

Da shell: `claude plugin install <nome>@<marketplace>`.

15.3 Interfaccia

<code>/theme</code>	Cambia tema colore (chiaro/scuro/automatico/daltonico/temi personalizzati)
<code>/statusline</code>	Configura la barra di stato
<code>/keybindings</code>	Apri il file delle scorciatoie da tastiera personalizzate
<code>/terminal-setup</code>	Configura Shift+Enter e altre scorciatoie nei terminali che lo richiedono (VS Code, Cursor, Alacritty, Zed)
<code>/config [chiave=valore]</code>	Apri le impostazioni, oppure imposta direttamente una chiave (da v2.1.181). Alias: <code>/settings</code>

Fonte: code.claude.com/docs/en/commands

16 Claude Code in VS Code

L'estensione VS Code è il metodo consigliato da Anthropic per usare Claude Code dentro l'IDE: motore, modelli, `CLAUDE.md`, hook, MCP e skill sono gli stessi della CLI da terminale; cambia l'interfaccia, con diff visualizzati inline e revisione del piano come documento markdown.

16.1 Differenze pratiche rispetto alla CLI pura

Aspetto	Comportamento
Diff e modifiche	Visualizzazione side-by-side inline nella sidebar, non disponibile nella sola CLI
Modalità piano	Il piano viene aperto come documento markdown modificabile prima dell'esecuzione
Configurazione MCP	Gestibile da <code>/mcp</code> nel pannello chat, ma la configurazione dei server resta un'operazione da terminale
Cronologia condivisa	Estensione e CLI condividono la stessa cronologia: una conversazione iniziata nel pannello si riprende da terminale con <code>claude -resume</code>
PATH della shell	L'installazione dell'estensione non aggiunge <code>claude</code> al PATH della shell: il binario standalone va installato separatamente per usarlo da terminale

16.2 Comandi utili specifici per l'integrazione IDE

<code>/ide</code>	Gestisce le integrazioni IDE e ne mostra lo stato
<code>/terminal-setup</code>	Da eseguire una volta per registrare Shift+Enter come “nuova riga” invece di “invia” nel terminale integrato

[i] Nota

Per collegare un'istanza CLI aperta in un terminale esterno (non integrato) a VS Code, eseguire `/ide` dentro la sessione Claude Code.

Fonte: code.claude.com/docs/en/vs-code

17 Glossario

Termine	Significato
Skill	Prompt strutturato in <code>SKILL.md</code> , invocabile manualmente o autonomamente da Claude
Hook	Comando automatico eseguito su un evento del ciclo di vita della sessione
MCP	Model Context Protocol: standard apertosi per collegare Claude a tool e dati esterni
Subagent	Istanza Claude dedicata a un task specifico, eseguibile in foreground o background
Worktree	Copia isolata del repository git usata per lavorare su un branch senza toccare la directory principale
Checkpoint	Stato salvato automaticamente prima di ogni modifica di Claude, ripristinabile con <code>/rewind</code>
Auto Memory	Note che Claude scrive autonomamente tra sessioni, distinte dal <code>CLAUDE.md</code> scritto a mano
Tool search	Meccanismo che carica le definizioni dei tool MCP solo quando servono, per non saturare il contesto

18 Fonti consultate

Tutte le informazioni in questo documento sono state verificate sulla documentazione ufficiale Anthropic, consultata a giugno 2026, alle pagine seguenti di code.claude.com/docs/en/: `commands`, `cli-reference`, `claude-directory`, `memory`, `skills`, `hooks`, `mcp`, `code-review`, `permission-modes`, `agent-view`, `vs-code`, `model-config`.

[!] Versione e cambiamenti

Claude Code rilascia aggiornamenti frequenti: nomi, alias e disponibilità dei comandi possono cambiare tra una versione e l'altra. Prima di documentare un workflow critico, verificare con `/release-notes` o sulla pagina ufficiale code.claude.com/docs/en/changelog.